



# Week 14 (DeadLock)

## ✓ Deadlock Avoidance

- Resource-allocation graph scheme

## ✓ Deadlock Detection

- Single Instance of Each Resource Type (Algo)
- Recovery from Deadlock: Process Termination

## | Content

## ✓ Deadlock Avoidance

- last week, we talk about deadlock, and we know that deadlock will occur when we have 4 conditions occur at the same time → that why OS didn't do anything about deadlock
- if OS want to do that → deadlock prevention → OS try to prevent 1 of 4 condition
  - the problem is the utilization of the resources are not good.
- If OS doesn't do prevention, how about avoidance

## | The system always in safe state → deadlock never happen

- the point is if a process request for a resource, and if OS give the resource to that process, and make the system unsafe → OS won't give resource.

## Basic Facts

- If a system is in safe state  $\Rightarrow$  no deadlocks.
- If a system is in unsafe state  $\Rightarrow$  possibility of deadlock.
- Avoidance  $\Rightarrow$  ensure that a system will never enter an unsafe state.

### Limitation of Deadlock avoidance

## Deadlock Avoidance

Requires that the system has some additional *a priori* information available.

- Simplest and most useful model requires that each process declare the *maximum number* of resources of each type that it may need.
- The deadlock-avoidance algorithm dynamically examines the resource-allocation state to ensure that there can never be a circular-wait condition.
- Resource-allocation state is defined by the number of available and allocated resources, and the maximum demands of the processes.

1. Each process must tell before it start
  - tell os all the resource it want to use  $\rightarrow$  and can't change after it start
2. OS will use this info to decide whether to give resource to the process (deadlock avoidance)
  - algorithm that support this

# Avoidance algorithms

- Single instance of a resource type. Use a resource-allocation graph
- Multiple instances of a resource type. Use the banker's algorithm

- we learn only the first one

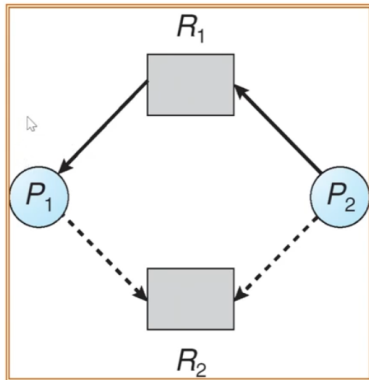
## Resource-Allocation Graph Scheme

### Resource-Allocation Graph Scheme

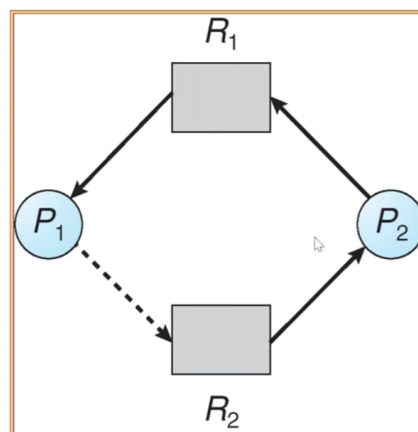
- *Claim edge*  $P_i \rightarrow R_j$  indicated that process  $P_i$  may request resource  $R_j$ ; represented by a dashed line.
- Claim edge converts to request edge when a process requests a resource.
- Request edge converted to an assignment edge when the resource is allocated to the process.
- When a resource is released by a process, assignment edge reconverts to a claim edge.
- Resources must be claimed *a priori* in the system.

- used resource-allocation graph, and add claim edge

## Resource-Allocation Graph



- 2 resources  $\rightarrow R_1, R_2$
- 2 processes  $\rightarrow P_1, P_2$
- in this system, at this moment,  $P_1$  hold  $R_1$ 
  - $P_2$  waiting/request for  $R_1$
- claim edge state that  $P_1$  will ask for  $R_2$ , and  $P_2$  will ask for  $R_2$  as well.
- This system is in safe state  $\rightarrow$  no cycle
- What if  $P_2$  request for  $R_2$ , if  $P_2$  request for  $R_2$ 
  - OS will let try first. change claim edge to assign edge



- and then, it kind of detect cycle  $\rightarrow$  OS said no, I won't give  $R_2$  to  $P_2 \rightarrow$  go back to the previous one.

- OS then try P1, and its okay → no cycle.
- and OS need to do this everytime process request for resources → not really work → OS like I won't do this
- so avoidance not really work.

## ✓ Deadlock Detection

- concept → OS allow deadlock to occur, detect and recovered

## Deadlock Detection

- Allow system to enter deadlock state
- Detection algorithm
- Recovery scheme

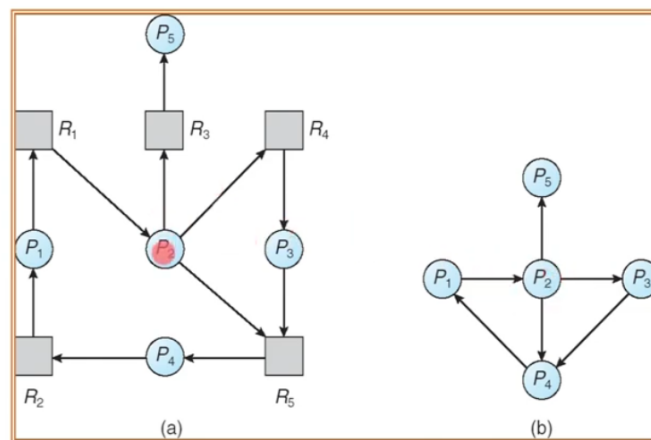
## 📌 Single Instance of Each Resource Type (Algo)

## Single Instance of Each Resource Type

- Maintain *wait-for* graph
    - Nodes are processes.
    - $P_i \rightarrow P_j$  if  $P_i$  is waiting for  $P_j$ .
  - Periodically invoke an algorithm that searches for a cycle in the graph. If there is a cycle, there exists a deadlock.
- An algorithm to detect a cycle in a graph requires an order of  $n^2$  operations, where  $n$  is the number of vertices in the graph.

- wait-for graph
  - convert resource allocation to wait-for graph

## Resource-Allocation Graph and Wait-for Graph



Resource-Allocation Graph      Corresponding wait-for graph

- find the direction from process to process → then draw an arrow.
- after we have wait-for graph
- os gonna detect the cycle in wait-for graph
  - os detect cycle  $p_1, 2, 3, 4 \rightarrow$  this 4 are in deadlock
  - os know that there deadlock occur in the system
  - if OS detect cycle → OS need to recover, but how

## Recovery from Deadlock: Process Termination

## Recovery from Deadlock: Process Termination

- ❑ Abort all deadlocked processes.
  - ❑ Abort one process at a time until the deadlock cycle is eliminated.
  - ❑ In which order should we choose to abort?
    - ❑ Priority of the process.
    - ❑ How long process has computed, and how much longer to completion.
    - ❑ Resources the process has used.
    - ❑ Resources process needs to complete.
    - ❑ How many processes will need to be terminated.
    - ❑ Is process interactive or batch?
- 
- one way is to do the process termination
    - os terminate process
    - the point is how to terminate the process
1. (not a good one) after detect deadlock → terminate all deadlocked process (p1-p4)
    - this 4 will never finish?
  2. (a better idea is) Abort one process at a time until the deadlock cycle is eliminate
    - let os choose, and let some process finish
    - but which condition it should use to select which process to be terminates
      - some
        - priority of the process → terminate lower priority (higher → more priority)
        - How long process has computed, and how much longer to completion → terminate the process that just started (process that run a long time → it consumed lot of resource already, and alos might finish in a short time)

### another way is

3. not terminate the process, but preemp the resource

- bring resource back for some process
  - but actually some resource can't get back
    - if some process always a victim → starvation may exist.

## Recovery from Deadlock: Resource Preemption

- Selecting a victim – minimize cost.
- Rollback – return to some safe state, restart process for that state.
- Starvation – same process may always be picked as victim, include number of rollback in cost factor.

### Quiz

1. If a cycle is detected in a resource-allocation graph then deadlock
  - False
2. The system is safe if the cycle is not detected after changing the claim edge to the assignment edge
  - True
3. An edge in a resource-allocation graph for deadlock avoidance that has a direction from a process to a resource to tell an OS that it may request for the resource is called \_\_\_\_
  - claim edge
4. Only one process at a time can use a resource
  - Mutual exclusion
5. Request edge of a resource-allocation graph has a direction from a resource to a process
  - False
6. Deadlock involves 2 or more processes
  - True
7. The claim edge will be changed to \_\_\_\_ to test for the safe state.



- Assignmetn edge
8. Resource-Allocation Graph that is used to model deadlock is an undirected graph
    - False
  9. A resource can be released only voluntarily by the process holding it
    - No-preemptive
  10. Usually, the operating system that we use today do not handle deadlock.
    - True
  11. Giving a number to each resource and requirning a process to request resources in an increasing order can prevent o preemption condition.
    - False
  12. Using a resource-allocation graph for deadlock avoidance can work if there is only one resource for each type of resource.
    - True
  13. From the C program in the picture, it is a deadlock-free program

## Question Image

```
C q17.c x Settings
C q17.c > main()
1  #include <pthread.h>
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <unistd.h>
5  #include <semaphore.h>
6  sem_t mutex1;
7  sem_t mutex2;
8  void *func1 (void *param);
9  void *func2(void *param);
10 int main() {
11     int i = 0;
12     pthread_t tid1, tid2;
13     pthread_attr_t attr;
14     pthread_attr_init(&attr);
15     sem_init(&mutex1, 0,1);
16     sem_init(&mutex2, 0,1);
17     pthread_create(&tid1, &attr, func1, NULL);
18     pthread_create(&tid2, &attr, func2, NULL);
19     pthread_join(tid1, NULL);
20     pthread_join(tid2, NULL);
21     sem_destroy(&mutex1);
22     sem_destroy(&mutex2);
23 }
24 void *func1 (void *param) {
25     sem_wait(&mutex1);
26     sem_wait(&mutex2);
27     printf("Hello\n");
28     sem_post(&mutex1);
29     sem_post(&mutex2);
30     pthread_exit(0);
31 }
32 void *func2 (void *param) {
33     sem_wait(&mutex2);
34     sem_wait(&mutex1);
35     printf("Hello\n");
36     sem_post(&mutex2);
37     sem_post(&mutex1);
38     pthread_exit(0);
39 }
```

**Q. From the C program in the picture, It is a deadlock-free program.**

- False

14. A process holding at least one resource and waiting for resource held by other processes
- Hold and wait
15. If only one condition in the 4 conditions for deadlock is true then deadlock occurs.
- False
16. One condition for deadlock avoidance is that a process must inform an OS of all resources it needs to use to finish its task before it starts.
- True
17. Deadlock handling mechanism(s) that does not allow a deadlock to occur
- deadlock avoidance and deadlock prevention
  - not deadlock detection